UDESC



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

UDESC

Array

Um arranjo de tamanho fixo determinado em tempo de compilação:

```
fn foo (a:&mut [i32]) // Slice
{
   for n in a {
      *n += 1;
   }
}

fn main() {
   let mut a = [1,2,3,4,5,6,7,8,9];
   a[1] = 200;
   foo(&mut a);
   println!("{a:?}");
}
```

UDESC

Vector

Um *vector* Vect<T> é um arranjo com tipo polimórfico T (paramétrico) que pode ser redimensionado:

```
fn foo (a:&mut Vec<i32>)
{
   for n in a {
      *n += 1;
   }
}

fn main() {
   let mut v1 = vec![1,2,3,4,5,6,7,8,9];
   let mut v2:Vec<i32> = (0..20).collect();

   foo(&mut v1);
   v2[0] = 1000;
   println!("{v1:?}");
   println!("{v2:?}");
}
```

UDESC

Slice

Um *slice* definido como [T] sem especificar o tamanho é uma região de um *array* ou de um *vector*. Um *slice* pode ter qualquer tamanho, sendo um ponteiro melhorado que além do endereço armazena o tamanho:

```
fn main() {
   let a = [1,2,3,4,5,6,7,8,9];
   let v:Vec<u32> = (1..20).collect();
   let s1 = &a[1..3];
   let s2 = &v[5..10];
   println!("{s1:?}");
   println!("{s2:?}");
}
```

```
[2, 3]
[6, 7, 8, 9, 10]
```

UDESC

Slice

```
fn main() {
   let mut a = [1,2,3,4,5,6,7,8,9];
   let mut v:Vec<u32> = (0..20).collect();
   let s1 = &mut a[1..3];
   let s2 = &mut v[5..10];

   s1[0] = 200;
   s2[1] = 200;
   println!("{v:?}");
   println!("{a:?}");
}
```

```
[0, 1, 2, 3, 4, 5, 200, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
[1, 200, 3, 4, 5, 6, 7, 8, 9]
```

UDESC

Slice

```
fn foo (a:&mut [i32])
{
   for n in a {
      *n += 1;
   }
}

fn main() {
   let mut a = [1,2,3,4,5,6,7,8,9];
   let mut v:Vec<i32> = (0..20).collect();

   foo(&mut a);
   foo(&mut v);
   println!("{v:?}");
   println!("{a:?}");
}
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
[2, 3, 4, 5, 6, 7, 8, 9, 10]
```

UDESC

Option

Option é um tipo enumerável que pode assumir uma de duas alternativas: None ou Some (T). O tipo *Option* deve ser usado, por exemplo, quando uma função pode ou não retornar a um valor.

```
Enum Option<T> {
   None,
   Some (T)
}
```

```
fn primeiro(a:&[i32])-> Option<i32> {
   if a.is_empty() {None}
   else {Some (a[0])}
}
```

```
fn main() {
   let a:[i32;0] = [];
   let r = primeiro(&a);

   println!("{r:?}");
}
```

UDESC

Pattern Match
(Casamento de padrão)

```
fn primeiro(a:&[i32])-> Option<i32> {
  if a.is_empty() {None}
  else {Some (a[0])}
}

fn main() {
  let a = [];
  match primeiro(&a) {
    None     => {println!("Vazio");}
    Some (x) => {println!("{x}");}
  }
}
```

UDESC

Pattern Match
(Casamento de padrão)

```
fn primeiro(a:&[i32])-> Option<i32> {
  if a.is_empty() {None}
  else {Some (a[0])}
}

fn main() {
  let a = [];
  let r = match primeiro(&a) {
    None     => "Vazio".to_string(),
    Some (x) => format!("{x}")
  };
  println!{"{r}"};
}
```

UDESC

Desempacotar

```
fn primeiro(a:&[i32])-> Option<i32> {
  if a.is_empty() {None}
  else {Some (a[0])}
}

fn main() {
    let a = [42];
    let r = primeiro(&a).unwrap();
    println!("{r}");
}
```